EXPERIMENT 8

Write a Python program to implement DFS

Register Number: **192511013**

192511013
192511013
192511013

Aim

To write a Python program to implement the Depth First Search (DFS) algorithm, which explores a graph by going as deep as possible along each branch before backtracking, using both recursion and an explicit stack.

Algorithm

Step 1: Represent the graph as an adjacency list.

Step 2: Recursive DFS: mark the current node as visited and add it to the traversal order.

Step 3: For each neighbour of the current node that is not yet visited, call DFS recursively on that neighbour.

Step 4: When all neighbours have been explored, the function returns, which backtracks to the previous node.

Step 5: Iterative DFS: push the start node onto a stack. While the stack is not empty, pop a node; if it is not visited, mark it visited, record it, and push its unvisited neighbours (in reverse order so the first neighbour is processed first).

Step 6: To find a path, extend the current path at each call and return it as soon as the goal node is reached.

Step 7: Print the DFS traversal order from both versions and the path found.

192511013

Program

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

def dfs_recursive(graph, node, visited=None, order=None):
    if visited is None:
        visited, order = set(), []
    visited.add(node)
    order.append(node)
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs_recursive(graph, neighbour, visited, order)
    return order

def dfs_iterative(graph, start):
    visited, order = set(), []
    stack = [start]
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            order.append(node)
            # reversed so that the first neighbour is explored first
            for neighbour in reversed(graph[node]):
                if neighbour not in visited:
                    stack.append(neighbour)
    return order

def dfs_path(graph, start, goal, path=None, visited=None):
    path = (path or []) + [start]
    visited = visited or set()
    visited.add(start)
    if start == goal:
        return path
    for neighbour in graph[start]:
        if neighbour not in visited:
            result = dfs_path(graph, neighbour, goal, path, visited)
            if result:
                return result
    return None

if __name__ == "__main__":
    print("Graph (adjacency list):")
    for node, adj in graph.items():
        print(f"  {node} -> {adj}")
    print("\nDFS traversal (recursive) from A:", ' -> '.join(dfs_recursive(graph, 'A')))
    print("DFS traversal (iterative) from A:", ' -> '.join(dfs_iterative(graph, 'A')))
    print("A path from A to F found by DFS :", ' -> '.join(dfs_path(graph, 'A', 'F')))
```

192511013

Output

```
Graph (adjacency list):
  A -> ['B', 'C']
  B -> ['A', 'D', 'E']
  C -> ['A', 'F']
  D -> ['B']
  E -> ['B', 'F']
  F -> ['C', 'E']

DFS traversal (recursive) from A: A -> B -> D -> E -> F -> C
DFS traversal (iterative) from A: A -> B -> D -> E -> F -> C
A path from A to F found by DFS : A -> B -> E -> F
```

192511013